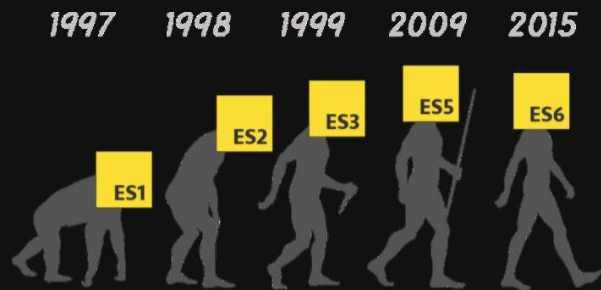


Modern JavaScript

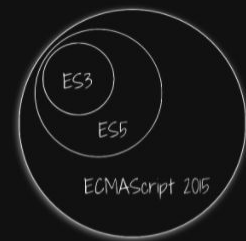


ES6/ES2015

ES6

ES6 is not just a modest set of new APIs but a radical jump!

- Constants
- Scoping
- Arrow Functions
- Extended Parameter Handling
- Template Literals
- Extended Literals
- Enhanced Regular Expression
- Enhanced Object Properties
- Destructuring Assignment
- Modules
- Classes
- Symbol Type
- Iterators
- Generators
- Map/Set/WeakMap/WeakSet
- Typed Arrays
- New Built-In Methods
- Promises
- Meta-Programming
- Internationalization & Localization



ES Next generation

Despite an impressive set of new features,
ES6 is backwards compatible

ES6 - New Features: Overview & Comparison

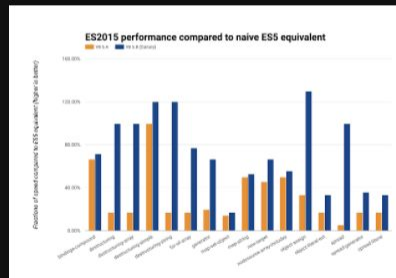
Browser Support



ES6 support varies substantially across browsers and JS environments

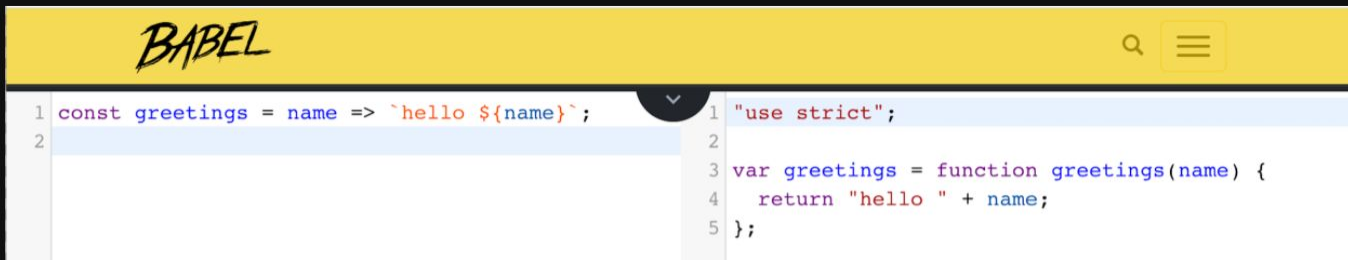


ES6 performance is getting better, but still catching up...



BABEL

Use ES6+ syntax without waiting for browser support



Transpiling is the process of compiling
ES6 source code into ES5 source code

Node Support



ES6 and ES2016(ES7) are already mainstream on Node.js

Nightly!

10.0.0	9.5.0	8.9.4	8.6.0	8.2.1	7.10.1	7.5.0	6.12.3	6.4.0	5.12.0	4.8.7	0.12.18	0.10.48
99% complete	99% complete	99% complete	99% complete	99% complete	99% complete	99% complete	99% complete	95% complete	59% complete	57% complete	31% complete	11% complete

Node.JS ES2015 support

Nightly!

10.0.0	9.5.0	8.9.4	8.6.0	8.2.1	7.10.1	7.5.0	6.12.3	6.4.0	5.12.0	4.8.7	0.12.18	0.10.48
100% complete	100% complete	100% complete	100% complete	100% complete	100% complete	100% complete	100% complete	76% complete	30% complete	7% complete	7% complete	0% complete

Node.JS ES2016 support

Block Scoped Declarations



In ES5, the fundamental unit of variable scoping was the function

```
var a = 2;  
(function iife() {  
  var a = 3;  
  console.log(a); // 3  
})();  
console.log(a); // 2
```

But in ES6 we can declare variables bounded to any code block

```
var a = 2;  
{  
  let a = 3;  
  console.log(a); // 3  
}  
console.log(a); // 2
```

No more reason to use function scope, block scoping is the way to go!

let



let declarations attach to the block scope

```
{  
  console.log(a); // undefined  
  var a = 1;  
  
  console.log(b); // ReferenceError thrown  
  let b = 2;  
}  
  
console.log(a); // 1  
console.log(b); // ReferenceError thrown
```

block scoped variables are not initialized until they appear in the block

let and closures



If used in the header of a **for** loop, **let** will create a new variable binding per loop iteration

```
var funcs = [];  
  
for (let i = 0; i < 5; i++) {  
  funcs.push(function() {  
    console.log(i);  
  });  
}  
  
funcs[1](); // 1  
funcs[3](); // 3
```

Closures created inside the loop iteration
close over each iteration binding

const



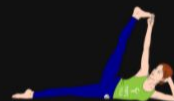
Read-only block scoped variable declaration

```
{  
  const a = 2;  
  console.log(a); // 2  
  
  a = 3; // TypeError!  
}
```

If the value of a constant is complex such as an object or an array, the objects themselves are mutable

```
{  
  const p = { x: 1, y: 0};  
  p.y = 1; // mutation of the referenced object is not a problem  
  
  p = { x: 1, y: 0}; // TypeError thrown  
}
```

... REST/SPREAD



When used as a function argument, it will gather the *rest* of the arguments into an array

```
function foo(a, ...args) {  
  console.log(a); // 1  
  console.log(args); // [2,3,4,5]  
}  
  
foo(1, 2, 3, 4, 5);
```

When used in front of an array it will *spread* out into its individual values

```
function foo(a, b, c, d, e, f) {  
  console.log(a, b, c, d, e, f);  
}  
  
const values = [1, 2, 3, 4, 5];  
  
foo(...values); // 1 2 3 4 5
```

No more need to use the `[].slice.call(arguments)` hack to convert arguments into an array

Default Parameter Values



A new syntax exists for setting a default value on a function parameter

```
// ES5 way
function foo(x, y) {
  x = x || 10;
  y = y || 30;
  return x + y;
}

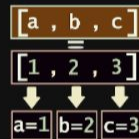
console.log(foo(5, 6)); // 11
console.log(foo(5)); // 35
console.log(foo(0, 1)); // oops... what is the problem here?

// ES6 way
function bar(x = 10, y = 30) {
  return x + y;
}

console.log(bar(5, 6)); // 11
console.log(bar(5)); // 35
console.log(bar(0, 1)); // 1
```

Much simpler and less error prone

Destructuring Assignment



Unpacks selected values from arrays, or properties from objects, into distinct variables

```
// ES5 style
var results, company, companyDetails;

results = Promise.all([
  companyService.get(id),
  companyService.getDetail(id)
]);
company = results[0];
companyDetails = results[1];
```

```
// ES6 style
const [company, companyDetail] = Promise.all([
  companyService.get(id),
  companyService.getDetail(id)
]);
```

Discarding non interesting properties from objects is easy

```
function getUser() {
  return { name: 'Rui', hobby: 'surf' };
}

let { name } = getUser(); // don't really care about hobbies...
console.log(name);
```

Object Enhancements



A number of important convenience extensions to the object literal exist in ES6

```
// ES5
var color = 'red';
var speed = '10';
var computedProp = 'hand brake';

var car = {
  color: color,
  speed: speed,
  start: function() {
    console.log('Vruuum!');
  }
};

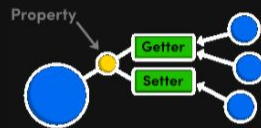
car[computedProp] = true;
```

```
// ES6
const color = 'red';
const speed = '10';
const computedProp = 'hand break';

let car = {
  color,
  speed,
  start() {
    console.log('Vruuum!');
  },
  [computedProp]: true
}
```

object enhancements help you write less code
and easily create complex Objects

Getters and Setters



Syntax to bind an object property to a function to be called when there is an attempt to get or set the property

```
let person = {  
  firstName: 'Rui',  
  lastName: 'Ferrão',  
  
  get fullName() {  
    return this.firstName + ' ' + this.lastName;  
  },  
  
  set fullName(value) {  
    [this.firstName, this.lastName] = value.split(' ');  
  }  
};  
  
console.log(person.fullName); // Rui Ferrão  
  
person.fullName = 'Pedro Antoninho';  
console.log(person.fullName); // Pedro Antoninho
```

Template Literals



String templates provide interpolation, giving us a lot more control over creating strings

```
// ES5
var name = 'Pedro';
var greeting = "Hello " + name + "!";

console.log(greeting); // Hello Pedro!
console.log(typeof greeting); // string
```

```
// ES6
let name = 'Pedro';
let greeting = `Hello ${name} !`;

console.log(greeting); // Hello Pedro!
console.log(typeof greeting); // string
```

A great benefit of interpolated string literals is they are allowed to span across multiple lines

```
let test = `Blessed is he who, in the name of charity and goodwill,
shepherds the weak through the valley of darkness,
for he is truly his brother's keeper, and the finder of lost children.
And I will strike down upon thee with great vengeance
and furious anger those who attempt to poison and destroy my brothers.`

console.log(text);
```

The line breaks in the interpolated string literal are preserved in the string value

Arrow Functions



Shortcut for creating anonymous functions with **this** bounded to lexical scope

```
// How can we fix this?
let person = {
  name: 'Rui',
  delayedHello(delay) {
    setTimeout(function() {
      console.log(this.name);
    }, delay);
  }
};

person.delayedHello(1000); // undefined
```

```
// ES6
let person = {
  name: 'Rui',
  delayedHello(delay) {
    setTimeout(() => {
      console.log(this.name);
    }, delay);
  }
};

person.delayedHello(1000); // Hello Rui!
```

Before ES6 we could solve this using **bind** to set the **this** context or grab the context on a closure (*that*)

The **arguments** object is not available with arrow functions, the **...rest** operator has to be used instead

Shorter Syntax

Arrow functions provide for a compact syntax



```
// ES5
var math = {
  product: function(x, y) {
    return x * y;
  },
  square: function(x) {
    return x * x;
  }
};
```

```
// ES6
let math = {
  product: (x, y) => x * y,
  square: x => x * x
}
```

we got rid of function brackets and return statement for one liners, as well as parentheses if the function receives only one argument

Classes



ES6 Classes are syntactic sugar over prototype-based inheritance

```
// ES5

// constructor function
function Person(name, age) {
  this.name = name;
  this.age = age;
}

// static method
Person.greet = function() {
  console.log('Hi!');
}

// instance method
Person.prototype.getFakeAge = function(years) {
  return this.age - years;
}
```

```
// ES6
class Person {

  constructor(name, age) {
    this.name = name;
    this.age = age;
  }

  static greet() {
    console.log('Hi!');
  }

  fakeAge(years) {
    return this.age - years;
  }
}
```

Subclassing



extends keyword can be used to create a class as a child of another class

```
class Animal {
  constructor(name) {
    this.name = name;
  }

  speak() {
    console.log(this.name + ' makes a noise.');
```

```
  }
}

class Dog extends Animal {
  speak() {
    console.log(this.name + ' barks.');
```

```
  }
}
```

It is now possible to extend natives such as **Array** or **Error**

Default Constructors

A default constructor is created for all classes if omitted



```
class Animal {  
    // default constructor for classes  
    constructor() { }  
};  
  
class Dog extends Animal {  
    // default constructor for subclasses  
    constructor(...args) {  
        super(...args);  
        // this is now available  
    }  
};
```

In a constructor of a subclass, **this** cannot be accessed until **super** is called

Modules



ES6 introduced a new standardized module format

- One module per file
- Module API is static
- Modules are singletons and maintain state
- Importing a module implies a blocking load

The mechanism for module loading is provided by the hosting environment (browser, node, webpack, etc.)

Current aligned Usage relative Date relative Show all									
IE	Edge *	Firefox	Chrome	Safari	iOS Safari *	Opera Mini *	Chrome for Android	UC Browser for Android	Samsung Internet
			49						
			62		10.2				
		57	63		10.3				4
11	16	58	64	11	11.2	all	64	11.8	6.2
	17	59	65	11.1	11.3				
		60	66	TP					
		61	67						

ES6 module loading is one of the latest feature to arrive to browsers

Module Exports



The **export** keyword can be used in front of a declaration or as an operator with a list of bindings to export

```
// all API members as named exports

export function add() { } // export add binding
export let multiplier = 100; // export multiplier binding

let odds = [1, 3, 5];
let evens = [2, 4, 6]
let zero = 0;
export { odds, evens }; // export odds and evens bindings
export zero as nil; // export nil as alias to zero binding

// one default export per module
export default function subtract() { } // default export
```

ES6 modules export immutable bindings, not values or references

Module Imports



The **import** keyword can be used to load a module from another one

```
// loads, compiles and evaluates the module without actually importing any of its bindings
import "module";

// import the default export as sub
import sub from 'module';

// import the named exports add and multiplier (aliased to mul)
import { add, multiplier as mul } from 'module';

// import the entire module API to the single math namespace
import * as math from 'module';
```

import only the specific bindings from a module that are required

Browser Module Loading



The latest browsers are already supporting module loading

```
// index.html
<script type="module">
  import {addTextToBody} from './utils.js';

  addTextToBody('Modules without babel/webpack!!!');
</script>
```

```
// utils.js
export function addTextToBody(text) {
  const div = document.createElement('div');
  div.textContent = text;
  document.body.appendChild(div);
}
```

type=module makes the browser treat the inline or external script as an ES6 module

Async/Await



Abstraction built on top of Promises/Generators that allows writing asynchronous code in a readable and synchronous looking fashion

```
// With ES6 Promises
function fetchUser(login) {

  const api = 'https://api.github.com/users';

  return fetch(`${api}/${login}`)
    .then(response => response.json())
    .then(user => ({
      name: user.name,
      location: user.location
    }));
}
```

```
// With ES2017 async/await
async function fetchUser(login) {

  const api = 'https://api.github.com/users';
  const response = await fetch(`${api}/${login}`);
  const user = await response.json();

  return {
    name: user.name,
    location: user.location
  };
}
```

async functions always return a Promise!
await schedules the execution of the remaining of the function
to when the promise is fulfilled!

Awaiting asynchronous functions



We can only **await** on an async function inside another async function

```
async function sleep(timeout) {  
  return new Promise((resolve) => setTimeout(resolve, timeout));  
}  
  
(async function() {  
  await sleep(1000); // wait for 1 second  
  console.log('Done!');  
})();
```

Top level **await** has to be wrapped in an **async** IIFE

```
// non-Promise values are wrapped in a Promise  
let value1 = (async () => { return await 42; })();  
let value2 = (async () => { return 42; })();  
  
console.log(value1 instanceof Promise); // true  
console.log(value2 instanceof Promise); // true
```

We can **await** on any thenable or non Promise value

Sequential Flows



Multiple promises can be executed sequentially using the **await** operator

```
async function fetchGitHub(endpoint) {  
  const api = `https://api.github.com/${endpoint}`;  
  
  const response = await fetch(api);  
  return response.json(); // return the promise, no need to await  
}  
  
async function showRepos(login) {  
  
  const user = await fetchGitHub(`users/${login}`);  
  const repos = await fetchGitHub(`users/${login}/repos`);  
  
  console.log(`${user.name} repositories:`);  
  repos.forEach((repo) => console.log(repo.name));  
}
```

How can the *showRepos* function be optimized?

Parallel Flows



The **Promise.all** method can be used to fire multiple promises in parallel

```
async function fetchGitHub(endpoint) {  
  const api = `https://api.github.com/${endpoint}`;  
  
  const response = await fetch(api);  
  return response.json(); // return the promise, no need to await  
}  
  
async function showRepos(login) {  
  const [user, repos] = await Promise.all([  
    fetchGitHub(`users/${login}`),  
    fetchGitHub(`users/${login}/repos`)  
  ]);  
  
  console.log(`${user.name} repositories:`);  
  repos.forEach((repo) => console.log(repo.name));  
}
```

As soon as one promise gets rejected, *Promise.all* will reject

Error Handling



Async functions return a rejected promise when an error is thrown

```
async function fetchGitHubUser(login) {  
  const api = `https://api.github.com/users/${login}`;  
  const response = await fetch(api);  
  const body = await response.json();  
  
  if (!response.ok) {  
    throw new Error(body.message); // throwing inside async rejects the returned promise  
  }  
  
  return body;  
}
```

try/catch blocks can be used to handle errors in asynchronous functions

```
async function showGitHubUser(login) {  
  try {  
    const user = await fetchGitHubUser(login);  
    console.log(`${user.name} is from ${user.location}`);  
  } catch (err) {  
    console.log(err.message);  
  }  
}
```

Looping with async/await



We can use async/await inside regular for loops

```
async function randomNums(howMany) {  
  let results = [];  
  for (let i = 0; i < howMany; i++) {  
    await sleep(1000);  
    results.push(Math.random());  
  }  
  return results;  
}  
  
console.log(await randomNums(10)); // 10 random numbers after 10 seconds
```

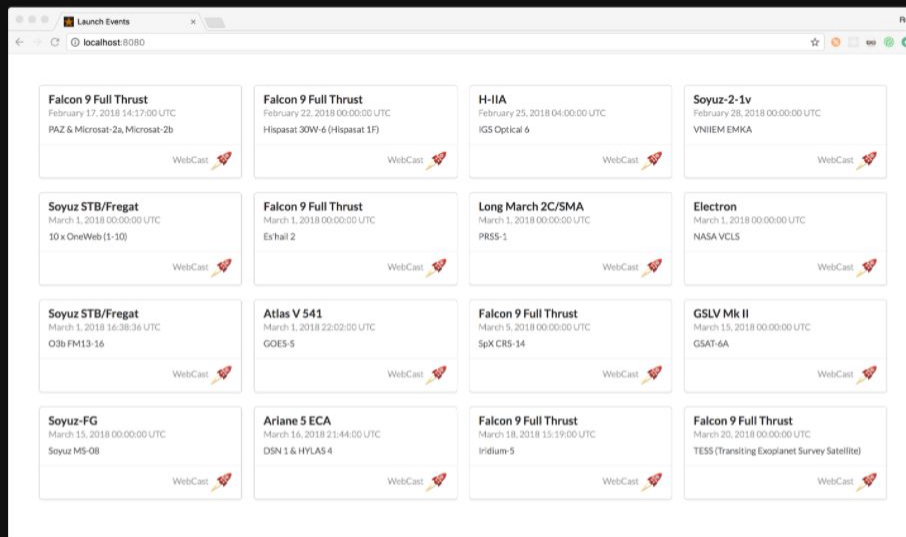
But things do not work quite as expected inside a **forEach** or other array methods...

```
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10].forEach(async (num) => {  
  await sleep(1000)  
  console.log(num)  
});  
  
console.log('Finished!');
```

What do you believe the output to be? Why?

{ Exercise }

AJAX with async/await



{ Exercise }

✓ you should get a green check mark on all tests

```
1. ferrao@Ruis-MacBook-Pro-3: ~/Dev/Workspace/academia-codigo/js-bootcamp/exercises/...  
✓ you should be able to use destructuring to discard non interesting properties  
✓ you should be able to destructure methods from native objects)  
  
getters and setters  
✓ you should be able to encapsulate a property behind a getter and a setter  
  
arrow functions  
✓ you should be able to bound this to lexical scope  
✓ you should be able to return the arguments as an array from inside an arrow function  
✓ you should be able to bound a function to the lexical scope  
✓ you should be able to bound to the lexical scope of an object created by a construct  
or function  
  
classes  
✓ you should be able to create a class with a constructor and a method  
✓ you should be able to subclass an existing class  
  
async/await  
✓ you should be able to use async/await for performing async operations  
  
75 passing (177ms)  
~/Dev/Workspace/academia-codigo/js-bootcamp/exercises/05-js-next/js-tests → master
```